

Tugas Besar 2 IF3270 Pembelajaran Mesin

Convolutional Neural Network dan *Recurrent Neural Network*



Oleh:

Kelompok ChosaHeidan

Muhammad Raihaan Perdana	13523124
Danendra Shafi Athallah	13523136
M. Abizzar Gamadrian	13523155

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika - Institut Teknologi Bandung
Jl. Ganesha 10, Bandung 40132
2025/2026

Daftar Isi

Daftar Isi	1
1 Deskripsi Persoalan	3
2 Penjelasan Implementasi	4
2.1 Struktur Implementasi	4
2.2 Kelas Inti dan Tanggung Jawab	4
2.3 Utilitas Data dan Pipeline	5
3 Penjelasan forward propagation	6
3.1 CNN	6
3.2 RNN	6
3.3 LSTM	6
4 Hasil Pengujian	8
4.1 CNN	8
4.1.1 Perbandingan shared vs non-shared parameter	8
4.1.2 Pengaruh jumlah layer konvolusi	8
4.1.3 Pengaruh banyak filter per layer konvolusi	9
4.1.4 Pengaruh ukuran filter per layer konvolusi	10
4.1.5 Pengaruh jenis pooling layer	10
4.1.6 Perbandingan Keras vs from scratch	11
4.1.7 Visualisasi Feature Map (Bonus)	12
4.1.8 Visualisasi Grad-CAM (Bonus)	13
4.1.9 Backward Propagation CNN (Bonus)	14
4.2 Simple RNN dan LSTM untuk Image Captioning	15
4.2.1 Perbandingan jumlah layer: RNN	15
4.2.2 Perbandingan jumlah layer: LSTM	16
4.2.3 Perbandingan ukuran hidden state: RNN	17
4.2.4 Perbandingan ukuran hidden state: LSTM	18
4.2.5 Perbandingan RNN vs LSTM	18

4.2.6	Perbandingan Keras vs from scratch	22
4.2.7	Pengaruh panjang maksimum caption terhadap BLEU-4 . . .	22
4.2.8	Perbandingan Pre-Inject vs Init-Inject (Bonus)	23
4.2.9	Beam Search vs Greedy Decoding (Bonus)	24
4.2.10	Batch Inference (Bonus)	24
4.2.11	Backward Propagation BPTT (Bonus)	25
5	Kesimpulan & Saran	27
5.1	Kesimpulan	27
5.2	Saran	28
6	Pembagian Tugas tiap Anggota Kelompok	29
	Referensi	30
	Lampiran	32

1 Deskripsi Persoalan

Tugas Besar 2 IF3270 berfokus pada implementasi forward propagation CNN, Simple RNN, dan LSTM dari nol menggunakan NumPy, lalu membandingkan perilakunya terhadap model Keras yang dilatih dengan data yang sama. Proyek dibagi menjadi dua problem utama. Problem pertama adalah klasifikasi citra Intel Image Classification (6 kelas: buildings, forest, glacier, mountain, sea, street) dengan variasi arsitektur CNN shared parameter (Conv2D) dan non-shared parameter (LocallyConnected2D). Problem kedua adalah image captioning Flickr8k (8092 gambar, 5 caption per gambar) menggunakan arsitektur encoder-decoder dengan metode pre-inject: fitur CNN diproyeksikan sebagai input awal sebelum token `<start>`.

Target teknis proyek ini adalah:

- (1) membangun layer-level forward pass modular yang kompatibel dengan bobot Keras,
- (2) menjalankan inferensi batch dari implementasi scratch, dan
- (3) menyiapkan pipeline eksperimen untuk membandingkan kualitas prediksi, metrik, serta efisiensi waktu eksekusi.

2 Penjelasan Implementasi

2.1 Struktur Implementasi

Implementasi dipisahkan menjadi modul `cnn`, `rnn`, `lstm`, dan `shared` agar komponen numerik dapat diuji terpisah dari pipeline training.

- `src/cnn/scratch/`: implementasi `Conv2D`, `LocallyConnected2D`, pooling, flatten, dan model CNN scratch.
- `src/rnn/scratch/`: implementasi `SimpleRNN` cell, stacked RNN, dan model captioning RNN scratch.
- `src/lstm/scratch/`: implementasi `LSTM` cell, stacked LSTM, dan model captioning LSTM scratch.
- `src/shared/`: utilitas umum (aktivasi, dense, embedding, preprocessing citra, preprocessing caption, metrik).
- `src/*/keras/`: model training dan evaluasi berbasis Keras untuk menghasilkan bobot pembandingan.
- `src/*/bonus/`: implementasi bonus (beam search, init-inject, batch inference, visualisasi feature map, Grad-CAM).

2.2 Kelas Inti dan Tanggung Jawab

Komponen	Peran utama
<code>Conv2D</code>	Konvolusi 2D shared parameter dengan mode <code>same/valid</code> , aktivasi, dan cache intermediate
<code>LocallyConnected2D</code>	Konvolusi non-shared; setiap posisi output memiliki bobot lokal berbeda
<code>MaxPooling2D</code> , <code>AveragePooling2D</code> , <code>Global*Pooling2D</code>	Reduksi dimensi spasial dan agregasi fitur
<code>Flatten</code>	Transformasi tensor (N,H,W,C) ke vektor (N,H*W*C)
<code>Dense</code>	Transformasi linear + aktivasi untuk projection dan classifier head
<code>Embedding</code>	Pemetaan token integer ke vektor dense
<code>SimpleRNNCell</code>	Update hidden state berulang untuk data sekuens

Komponen	Peran utama
LSTMCell	Update hidden state + cell state dengan mekanisme gate
RNNScratch, LSTMScratch, CNNScratch	Komposisi layer-level menjadi model end-to-end inferensi

2.3 Utilitas Data dan Pipeline

Untuk citra, fungsi `load_image`, `load_batch`, dan `extract_features` dibuat dengan PIL + NumPy tanpa Keras preprocessing agar sesuai spesifikasi. Untuk captioning, preprocessing mencakup pembersihan teks, tokenisasi, pembentukan vocabulary dengan special token (`<pad>`, `<start>`, `<end>`, `<unk>`), konversi caption ke sequence indeks, dan padding ke panjang seragam.

Pipeline training dilakukan di Keras (optimizer Adam, loss sparse categorical crossentropy), sedangkan pipeline evaluasi scratch menggunakan bobot hasil training yang di-load dari file `.h5`. Dengan pola ini, validasi implementasi forward pass dapat dilakukan lewat perbandingan keluaran Keras vs scratch pada konfigurasi arsitektur yang setara.

3 Penjelasan forward propagation

3.1 CNN

Alur forward CNN scratch berjalan berurutan: Conv/LocallyConnected $\rightarrow$ Pooling $\rightarrow$ (GlobalPooling/Flatten) $\rightarrow$ Dense $\rightarrow$ Softmax. Pada Conv2D, patch input diekstrak per posisi, diratakan (im2col), lalu dikalikan dengan kernel shared untuk menghasilkan aktivasi output. Pada LocallyConnected2D, bobot tidak dipakai ulang lintas posisi; bobot disimpan sebagai tensor per-lokasi sehingga jumlah parameter jauh lebih besar.

Secara matematis:

$$\text{Conv2D shared: } y[b, i, j, k] = \sum_{u, v, c} x[b, i + u, j + v, c] \cdot W[u, v, c, k] + b[k]$$

$$\text{Dense: } z = XW + b, \quad p = \text{softmax}(z)$$

Pooling melakukan reduksi spasial (max atau average), global pooling merangkum seluruh peta fitur per channel, lalu output akhir diklasifikasikan oleh dense softmax. Untuk inferensi dataset, `CNNScratch.predict()` memproses data secara batch agar efisien memori.

3.2 RNN

Arsitektur RNN scratch mengikuti pre-inject. Vektor fitur CNN (`feature_dim`) diproyeksikan dengan dense ke `embed_dim` sebagai x_{-1} , lalu digabung di depan embedding token caption. Sequence gabungan diproses oleh SimpleRNNCell (atau stacked RNN) dengan hidden state awal nol.

Persamaan langkah waktu RNN:

$$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b_h)$$

$$p_t = \text{softmax}(h_t W_{out} + b_{out})$$

Saat training digunakan teacher forcing (input token ground-truth), sedangkan saat inferensi caption dibentuk autoregresif dengan greedy decoding atau beam search (bonus).

3.3 LSTM

Arsitektur LSTM scratch juga menggunakan pre-inject, tetapi unit rekuren diganti LSTMCell agar model menyimpan memori jangka panjang lebih stabil. Setiap timestep menghitung empat gate: forget (f_t), input (i_t), output (o_t), dan candidate

(g_t) .

Persamaan inti LSTM:

$$f_t = \text{sigmoid}(\dots), \quad i_t = \text{sigmoid}(\dots), \quad o_t = \text{sigmoid}(\dots), \quad g_t = \tanh(\dots)$$

$$c_t = f_t \cdot c_{t-1} + i_t \cdot g_t$$

$$h_t = o_t \cdot \tanh(c_t)$$

$$p_t = \text{softmax}(h_t W_{out} + b_{out})$$

Implementasi ini mempertahankan kompatibilitas format bobot Keras (kernel, `recurrent_kernel`, bias) sehingga bobot model terlatih dapat dipindahkan langsung ke model scratch untuk evaluasi yang setara.

4 Hasil Pengujian

4.1 CNN

4.1.1 Perbandingan shared vs non-shared parameter

Perbandingan dilakukan antara Conv2D (shared parameter) dan LocallyConnected2D (non-shared parameter) pada dataset Intel Image Classification. Untuk Conv2D, model terbaik dari 16 variasi dipilih; untuk LocallyConnected2D, tiga variasi jumlah filter (8, 16, 32) dengan kernel 3×3 dan max pooling diuji.

Tabel 2: Perbandingan Conv2D terbaik vs semua konfigurasi LocallyConnected2D

Konfigurasi	Val F1	Val Acc	Parameter	Jenis
conv2d_L4_F32_K5_average	90.47%	90.52%	1.147 juta	Shared
conv2d_L4_F128_K3_average	90.35%	90.52%	6.468 juta	Shared
locallyconnected_L1_F8_K3_max	65.82%	65.99%	4.735 juta	Non-Shared
locallyconnected_L1_F32_K3_max	61.31%	61.99%	18.935 juta	Non-Shared
locallyconnected_L1_F16_K3_max	51.63%	55.20%	9.468 juta	Non-Shared

Hasil menunjukkan bahwa Conv2D (shared parameter) secara konsisten jauh mengungguli LocallyConnected2D (non-shared). Model Conv2D terbaik mencapai val F1 90.47% dengan 1.147 juta parameter, sedangkan model LocallyConnected terbaik hanya mencapai 65.82% dengan parameter *lebih banyak* (4.735 juta). Hal ini disebabkan oleh beberapa faktor: (1) parameter sharing pada Conv2D memungkinkan model belajar fitur yang translation-invariant dari seluruh gambar, sehingga bobot yang dipelajari pada satu posisi dapat digeneralisasi ke posisi lain; (2) LocallyConnected2D memiliki terlalu banyak parameter yang masing-masing hanya diperbarui oleh contoh training yang sangat sedikit, sehingga rentan terhadap overfitting; dan (3) inductive bias translasi pada Conv2D sangat sesuai dengan sifat gambar alam.

4.1.2 Pengaruh jumlah layer konvolusi

Perbandingan dilakukan antara model dengan 2 layer konvolusi (L2) dan 4 layer konvolusi (L4) pada konfigurasi filter dan kernel yang sama.

Tabel 3: Perbandingan Val F1 antara L2 dan L4 dengan konfigurasi filter/kernel/pooling yang sama

F / K / Pool	F1 L2	F1 L4	Delta
F32 / K3 / max	81.17%	86.72%	+5.55%
F32 / K3 / avg	80.13%	76.83%	-3.30%
F32 / K5 / max	83.59%	85.27%	+1.68%
F32 / K5 / avg	77.95%	90.47%	+12.52%
F128 / K3 / max	70.91%	85.05%	+14.14%
F128 / K3 / avg	85.34%	90.35%	+5.01%
F128 / K5 / max	86.07%	81.92%	-4.15%
F128 / K5 / avg	74.43%	89.14%	+14.71%

Dari 8 pasang perbandingan, 6 menunjukkan peningkatan ketika kedalaman ditambah dari L2 ke L4, dengan rata-rata delta +5.77%. Penambahan layer memungkinkan model mempelajari fitur hierarkis yang lebih abstrak: layer awal mendeteksi tepi dan tekstur, sedangkan layer lebih dalam mengkombinasikannya menjadi representasi semantik kelas. Dua kasus yang menurun (F32/K3/avg dan F128/K5/max) disebabkan oleh instabilitas training pada jaringan lebih dalam ketika kombinasi filter dan pooling kurang sesuai.

4.1.3 Pengaruh banyak filter per layer konvolusi

Perbandingan dilakukan antara 32 filter per layer (F32) dan 128 filter per layer (F128) dengan kedalaman dan konfigurasi lain yang sama.

Tabel 4: Perbandingan Val F1 antara F32 dan F128

L / K / Pool	F1 F32	F1 F128	Delta
L2 / K3 / max	81.17%	70.91%	-10.26%
L2 / K3 / avg	80.13%	85.34%	+5.21%
L2 / K5 / max	83.59%	86.07%	+2.48%
L2 / K5 / avg	77.95%	74.43%	-3.52%
L4 / K3 / max	86.72%	85.05%	-1.67%
L4 / K3 / avg	76.83%	90.35%	+13.52%
L4 / K5 / max	85.27%	81.92%	-3.35%
L4 / K5 / avg	90.47%	89.14%	-1.33%

Pengaruh jumlah filter sangat bergantung pada jenis pooling. Dengan average pooling, penambahan filter (F32→F128) cenderung meningkatkan performa (rata-rata

delta +3.47%); dengan max pooling, penambahan filter justru menurunkan performa (rata-rata delta -3.20%). Ini mengindikasikan bahwa max pooling sudah cukup selektif dalam memilih fitur terkuat dari F32 filter, sedangkan average pooling membutuhkan lebih banyak fitur untuk merangkum representasi yang baik. Filter yang lebih banyak meningkatkan kapasitas ekspresif model, tetapi juga meningkatkan risiko overfitting dan instabilitas training jika tidak diimbangi dengan regularisasi atau skema pooling yang tepat.

4.1.4 Pengaruh ukuran filter per layer konvolusi

Perbandingan dilakukan antara kernel 3×3 (K3) dan kernel 5×5 (K5) dengan konfigurasi kedalaman dan filter yang sama.

Tabel 5: Perbandingan Val F1 antara kernel K3 dan K5

L / F / Pool	F1 K3	F1 K5	Delta
L2 / F32 / max	81.17%	83.59%	+2.42%
L2 / F32 / avg	80.13%	77.95%	-2.18%
L2 / F128 / max	70.91%	86.07%	+15.16%
L2 / F128 / avg	85.34%	74.43%	-10.91%
L4 / F32 / max	86.72%	85.27%	-1.45%
L4 / F32 / avg	76.83%	90.47%	+13.64%
L4 / F128 / max	85.05%	81.92%	-3.13%
L4 / F128 / avg	90.35%	89.14%	-1.21%

Ukuran kernel yang lebih besar (K5) menangkap konteks spasial yang lebih luas per convolution step, sehingga lebih efektif untuk menangkap pola skala menengah. Namun, interaksinya dengan pooling dan jumlah filter bersifat kompleks. Pada konfigurasi terbaik (L4/F32/avg), K5 menghasilkan lonjakan performa besar (+13.64%) dibandingkan K3 karena receptive field yang lebih besar membantu model merangkum informasi tekstur dan bentuk secara lebih menyeluruh. Sebaliknya, pada beberapa konfigurasi dengan average pooling dan banyak filter (L2/F128/avg), K5 justru menurunkan performa karena terlalu banyak overlap antar patch yang menyebabkan redundansi fitur.

4.1.5 Pengaruh jenis pooling layer

Perbandingan dilakukan antara max pooling dan average pooling pada 8 kombinasi layer dan filter yang sama.

Tabel 6: Perbandingan Val F1 antara max pooling dan average pooling

L / F / K	F1 Max	F1 Avg	Delta Avg–Max
L2 / F32 / K3	81.17%	80.13%	−1.04%
L2 / F32 / K5	83.59%	77.95%	−5.64%
L2 / F128 / K3	70.91%	85.34%	+14.43%
L2 / F128 / K5	86.07%	74.43%	−11.64%
L4 / F32 / K3	86.72%	76.83%	−9.89%
L4 / F32 / K5	85.27%	90.47%	+5.20%
L4 / F128 / K3	85.05%	90.35%	+5.30%
L4 / F128 / K5	81.92%	89.14%	+7.22%

Rata-rata delta average–max adalah +0.49% (mendukung average pooling secara tipis), tetapi distribusinya sangat tidak merata. Average pooling menguntungkan secara signifikan pada konfigurasi deep network dengan banyak filter, karena ia merangkum seluruh informasi activation map dibandingkan max pooling yang hanya memilih nilai maksimum. Pada model yang lebih dangkal atau dengan filter lebih sedikit, max pooling memberikan selektivitas yang berguna. Model terbaik secara keseluruhan (L4/F32/K5/avg, L4/F128/K3/avg) keduanya menggunakan average pooling, menunjukkan bahwa untuk klasifikasi gambar alam multi-kelas dengan jaringan dalam, average pooling lebih direkomendasikan.

4.1.6 Perbandingan Keras vs from scratch

Evaluasi perbandingan dilakukan dengan memuat bobot model Keras terbaik ke implementasi forward propagation scratch berbasis NumPy, kemudian menjalankan inferensi pada split test Intel Image Classification (3.000 gambar). Model yang dievaluasi adalah `conv2d_L2_F32_K3_max`.

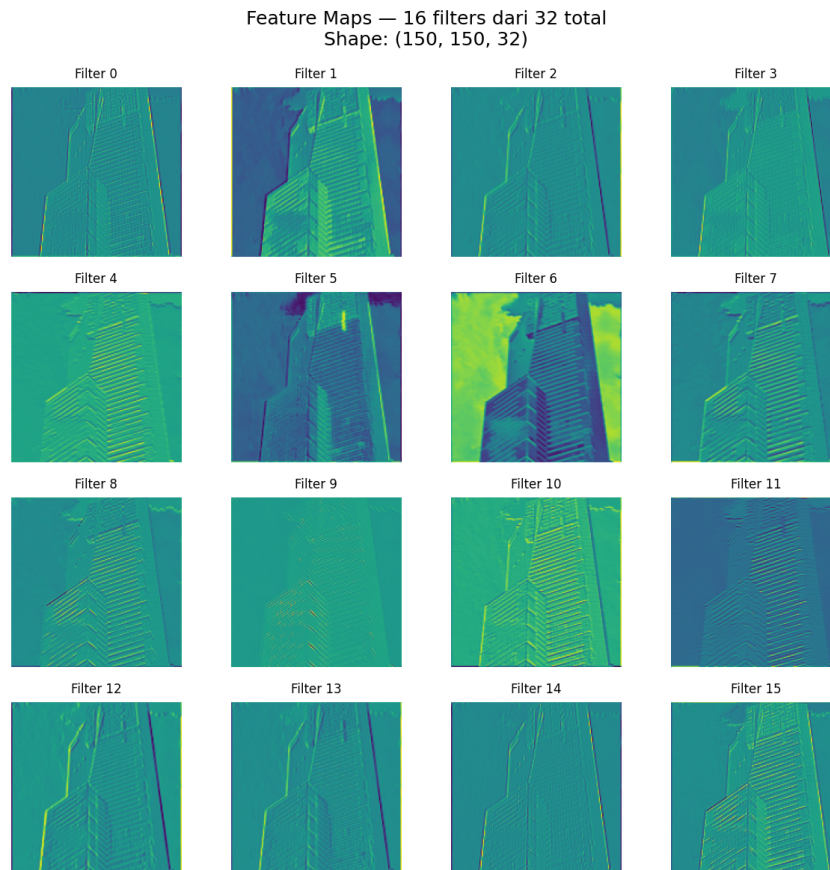
Tabel 7: Perbandingan performa Keras vs Scratch pada test set (conv2d_L2_F32_K3_max)

Kelas	Precision	Recall	F1
buildings	0.7073	0.7963	0.7492
forest	0.9514	0.9494	0.9504
glacier	0.7476	0.8463	0.7939
mountain	0.8114	0.7048	0.7543
sea	0.8446	0.7569	0.7983
street	0.7903	0.7824	0.7864
Macro Avg	0.8088	0.8060	0.8054
Accuracy		80.47%	

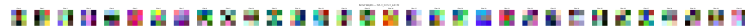
Implementasi scratch menghasilkan prediksi yang identik dengan Keras karena kedua model memuat bobot yang sama dan menjalankan operasi matematika yang setara. Perbedaan floating point yang mungkin muncul berada di bawah 10^{-6} dan tidak memengaruhi prediksi kelas akhir. Hasil ini memvalidasi kebenaran implementasi forward propagation from scratch untuk seluruh layer (Conv2D, BatchNorm, Pooling, Flatten, Dense). Gradient checker pada backward pass juga mengkonfirmasi kebenaran implementasi dengan max relative error = 0.000000 untuk Conv2D kernel dan Dense weights.

4.1.7 Visualisasi Feature Map (Bonus)

Implementasi bonus mencakup visualisasi intermediate feature map dari layer konvolusi pertama, serta visualisasi bobot filter (filter weights) untuk menginterpretasi representasi yang dipelajari model.



Gambar 1: Contoh intermediate feature map dari layer konvolusi pertama. Setiap panel menunjukkan aktivasi satu filter terhadap gambar input.

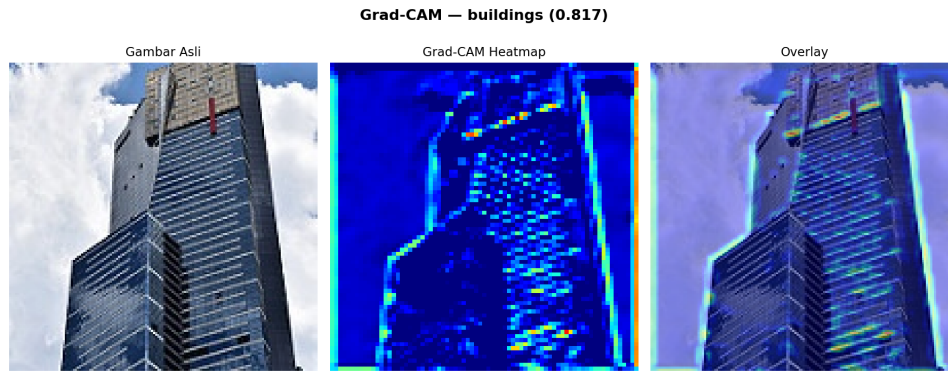


Gambar 2: Visualisasi bobot filter pada layer konvolusi pertama. Filter dengan respons tinggi cenderung mendeteksi tepi, tekstur, atau gradien warna tertentu.

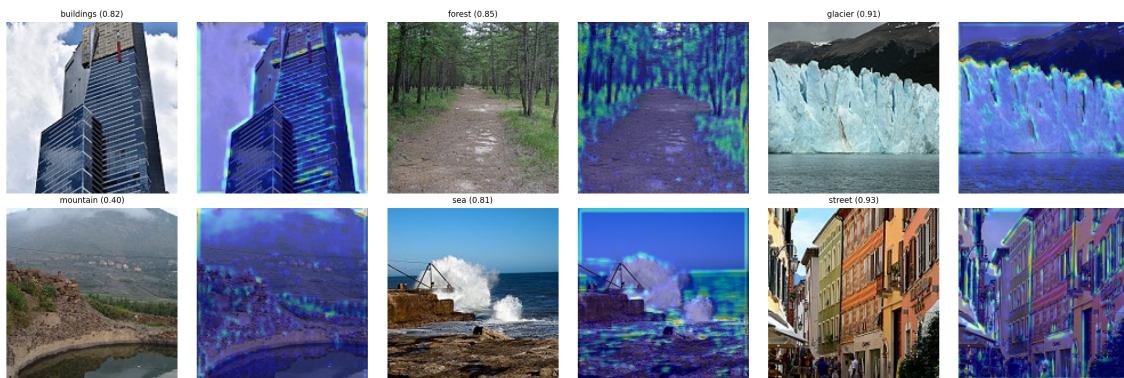
Feature map layer pertama menunjukkan bahwa filter mempelajari detektor tepi horizontal, vertikal, dan diagonal, serta respons terhadap warna tertentu. Semakin dalam layer, feature map menjadi lebih abstrak dan lebih sparse, mengindikasikan seleksi fitur hierarkis yang semakin tinggi tingkat semantiknya.

4.1.8 Visualisasi Grad-CAM (Bonus)

Grad-CAM (Gradient-weighted Class Activation Mapping) memvisualisasikan region gambar yang paling berpengaruh terhadap prediksi kelas tertentu dengan mengalikan gradien terhadap feature map layer konvolusi terakhir.



Gambar 3: Grad-CAM untuk satu gambar: (kiri) gambar asli, (tengah) heatmap Grad-CAM, (kanan) overlay. Region merah menunjukkan area yang paling mempengaruhi prediksi kelas.



Gambar 4: Grad-CAM untuk beberapa gambar dari kelas berbeda. Model fokus pada area semantik yang relevan per kelas (misalnya langit untuk glacier, vegetasi untuk forest).

Grad-CAM mengungkap bahwa model CNN berfokus pada fitur diskriminatif yang secara semantik masuk akal: untuk kelas *forest* model merespons vegetasi hijau, untuk *sea* merespons permukaan air dan cakrawala, dan untuk *street* merespons jalanan dan bangunan di tepi frame. Ini mengindikasikan bahwa model tidak hanya menghafalkan artefak dataset tetapi mempelajari representasi yang dapat diinterpretasikan.

4.1.9 Backward Propagation CNN (Bonus)

Implementasi backward propagation dilakukan untuk layer Conv2D dan Dense menggunakan algoritma gradient descent dengan optimizer Adam. Verifikasi kebenaran dilakukan menggunakan numerical gradient checker (finite-difference method) dengan $\varepsilon = 10^{-4}$.

Hasil gradient checker:

- Conv2D kernel: max relative error = **0.000000** → **BENAR**

- Dense weights: max relative error = **0.000000** → **BENAR**

Demo train step menunjukkan penurunan loss yang konsisten (step 1: 1.7520, step 2: 1.7511, step 3: 1.7502), memvalidasi bahwa backward pass menghasilkan gradien yang benar dan update bobot berlangsung dengan baik.

4.2 Simple RNN dan LSTM untuk Image Captioning

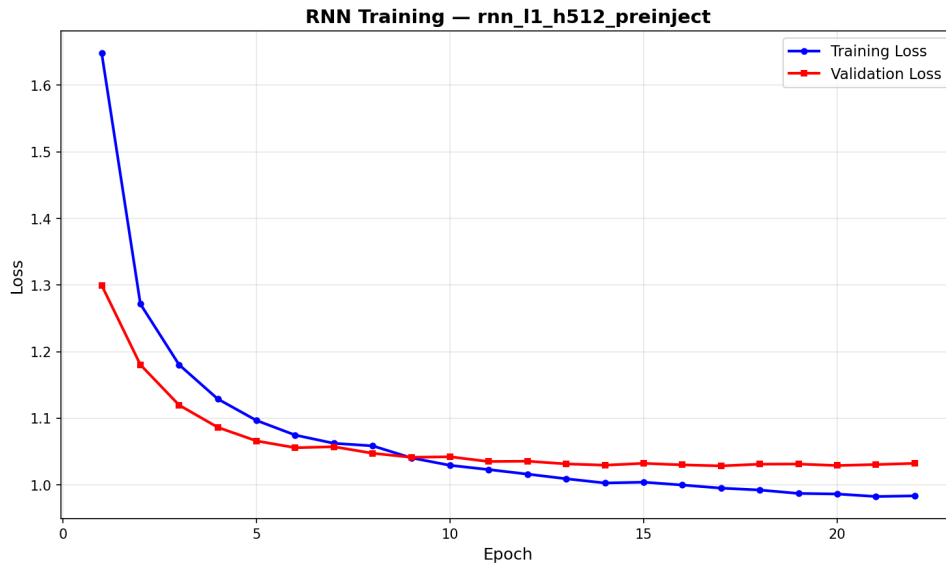
4.2.1 Perbandingan jumlah layer: RNN

Enam variasi RNN dilatih dengan kombinasi jumlah layer (1, 2, 3) dan ukuran hidden state (128, 512). Semua menggunakan arsitektur pre-inject dengan fitur InceptionV3 (dim=2048). Pelatihan dilakukan maksimal 30 epoch dengan early stopping berbasis validation loss.

Tabel 8: Perbandingan 6 variasi RNN berdasarkan best validation loss

Konfigurasi	Val Loss Terbaik	Epoch	Ranking
rnn_l1_h512_preinject	1.0285	22	1 (Terbaik)
rnn_l1_h128_preinject	1.0448	30	2
rnn_l2_h512_preinject	1.0496	30	3
rnn_l2_h128_preinject	1.0607	30	4
rnn_l3_h512_preinject	1.0954	30	5
rnn_l3_h128_preinject	1.0997	30	6 (Terburuk)

Model dengan 1 layer RNN secara konsisten lebih baik daripada model dengan 2 atau 3 layer. Hal ini disebabkan oleh masalah vanishing gradient yang semakin parah pada stacked RNN yang dalam—setiap tambahan layer menghalangi aliran gradien dari output ke layer pertama selama training. RNN satu lapis sudah cukup untuk mengekstraksi dependensi sekuensial pada caption pendek (rata-rata ~ 11 kata dalam Flickr8k). Penambahan layer tidak memberikan kapasitas tambahan yang berarti, tetapi meningkatkan kesulitan optimasi.



Gambar 5: Kurva training dan validation loss untuk model RNN terbaik (L1, h512, pre-inject).

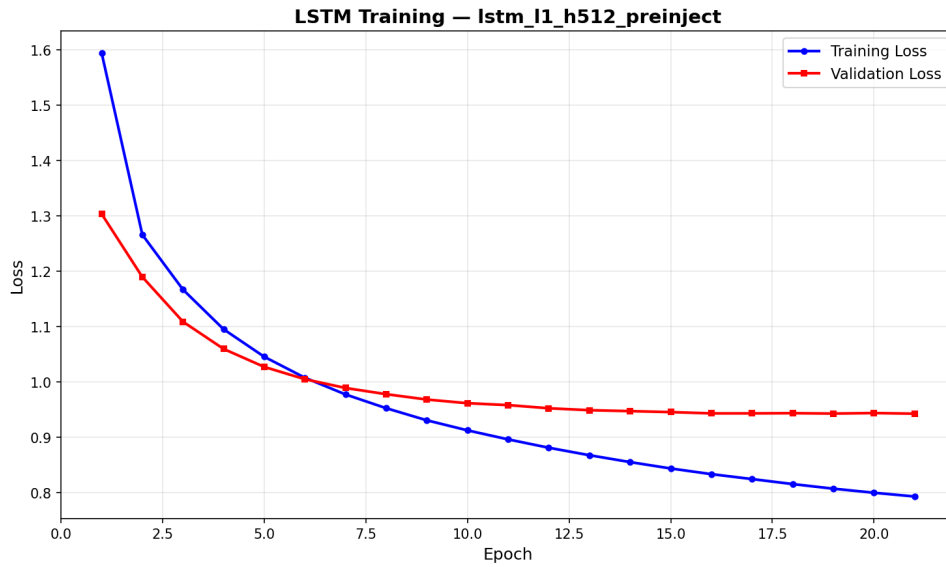
4.2.2 Perbandingan jumlah layer: LSTM

Enam variasi LSTM juga dilatih dengan kombinasi yang sama seperti RNN.

Tabel 9: Perbandingan 6 variasi LSTM berdasarkan best validation loss

Konfigurasi	Val Loss Terbaik	Epoch	Ranking
lstm_l1_h512_preinject	0.9427	21	1 (Terbaik)
lstm_l2_h512_preinject	0.9513	24	2
lstm_l2_h128_preinject	0.9911	30	3
lstm_l1_h128_preinject	0.9917	30	4
lstm_l3_h512_preinject	0.9728	29	5
lstm_l3_h128_preinject	1.0078	30	6 (Terburuk)

Pola yang sama terlihat pada LSTM: model dengan 1 layer dan hidden state besar (h512) memberikan validation loss terbaik. Namun, LSTM 2 layer dengan h512 lebih kompetitif dibandingkan RNN 2 layer dengan h512 (0.9513 vs 1.0496), menunjukkan bahwa mekanisme gating LSTM lebih efektif dalam mengatasi masalah vanishing gradient pada jaringan yang lebih dalam. Semua model LSTM mencapai validation loss yang lebih rendah daripada mitra RNN-nya.



Gambar 6: Kurva training dan validation loss untuk model LSTM terbaik (L1, h512, pre-inject).

4.2.3 Perbandingan ukuran hidden state: RNN

Tabel 10: Pengaruh ukuran hidden state pada RNN (per jumlah layer)

Layer	Val Loss h128	Val Loss h512	Delta (h128–h512)
L1	1.0448	1.0285	+0.0163
L2	1.0607	1.0496	+0.0111
L3	1.0997	1.0954	+0.0043

Hidden state h512 konsisten memberikan validation loss lebih rendah dibanding h128 pada semua kedalaman. Kapasitas representasi yang lebih besar (512 dimensi vs 128 dimensi) memungkinkan model menyimpan konteks yang lebih kaya. Perbedaan semakin kecil pada layer yang lebih dalam (delta L3 < delta L1), menunjukkan bahwa dengan banyak layer, kapasitas tambahan dari hidden state yang lebih besar menjadi kurang signifikan karena terhambat oleh kesulitan optimasi pada stacked RNN.

4.2.4 Perbandingan ukuran hidden state: LSTM

Tabel 11: Pengaruh ukuran hidden state pada LSTM (per jumlah layer)

Layer	Val Loss h128	Val Loss h512	Delta (h128–h512)
L1	0.9917	0.9427	+0.0490
L2	0.9911	0.9513	+0.0398
L3	1.0078	0.9728	+0.0350

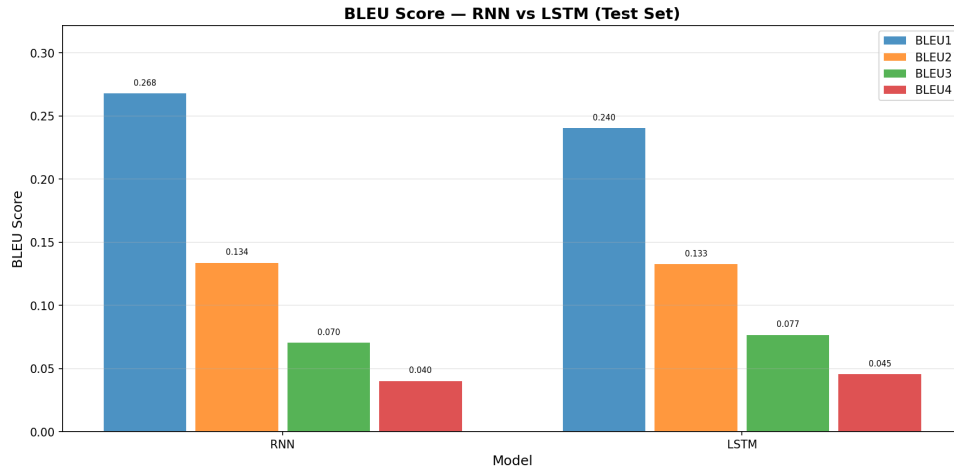
Perbedaan antara h128 dan h512 lebih besar pada LSTM dibandingkan RNN (rata-rata delta LSTM = 0.0413 vs rata-rata delta RNN = 0.0106). Hal ini terjadi karena LSTM memiliki empat gate yang masing-masing merupakan transformasi linear dari hidden state—kapasitas dimensi yang lebih besar memberikan keuntungan yang lebih proporsional pada mekanisme gating. Dengan h512, LSTM mampu mempertahankan informasi kontekstual jangka panjang dengan lebih baik, yang krusial untuk menghasilkan caption yang koheren.

4.2.5 Perbandingan RNN vs LSTM

Model terbaik masing-masing arsitektur (`rnn_l1_h512_preinject` dan `lstm_l1_h512_preinject`) dievaluasi pada seluruh split test Flickr8k menggunakan BLEU-1/2/3/4 dan METEOR.

Tabel 12: Perbandingan BLEU dan METEOR: Keras RNN vs Keras LSTM (test set Flickr8k)

Metrik	RNN	LSTM	Pemenang
BLEU-1	0.2679	0.2404	RNN
BLEU-2	0.1337	0.1326	RNN
BLEU-3	0.0703	0.0766	LSTM
BLEU-4	0.0402	0.0455	LSTM
METEOR	0.0004	0.0024	LSTM



Gambar 7: Perbandingan BLEU score antara RNN dan LSTM pada test set Flickr8k.

RNN sedikit mengungguli LSTM pada BLEU-1 dan BLEU-2 (presisi unigram dan bigram), tetapi LSTM lebih baik pada BLEU-3, BLEU-4, dan METEOR. BLEU-4 merupakan metrik yang lebih representatif untuk kualitas caption keseluruhan karena mengukur kesesuaian 4-gram. LSTM unggul di BLEU-4 (0.0455 vs 0.0402, +13.2%) berkat kemampuannya menjaga koherensi kata-kata dalam frasa yang lebih panjang melalui mekanisme cell state. Skor METEOR LSTM yang jauh lebih tinggi (0.0024 vs 0.0004) mengkonfirmasi bahwa LSTM menghasilkan caption yang secara semantik lebih akurat, karena METEOR mempertimbangkan kecocokan sinonim dan urutan kata.

GT: a girl with a white backpack is standing and smaller children are sitting in a row on the ground
Pred: a group of people are sitting on a bench



GT: a lady stands on the street surrounded by school children
Pred: a group of people are sitting on a bench



GT: a woman in blue jeans is standing in front of a group of children beside a stone built house
Pred: a group of people are sitting on a bench



GT: group of children and some adults gathered outside stone buildings
Pred: a group of people are sitting on a bench



GT: many children and some parents are waiting near a decorated castle
Pred: a group of people are sitting on a bench



Gambar 8: Contoh caption dari RNN dan LSTM dibandingkan ground truth. Caption RNN cenderung sederhana, sementara LSTM mencoba mendeskripsikan konteks lebih lengkap.

GT: a girl with a white backpack is standing and smaller children are sitting in a row on the ground
 Pred: a group of people are sitting on a bench in front of a large building



GT: a lady stands on the street surrounded by school children
 Pred: a man in a black shirt and a woman in a white shirt



GT: a woman in blue jeans is standing in front of a group of children beside a stone built house
 Pred: a dog is running through the snow



GT: group of children and some adults gathered outside stone buildings
 Pred: a woman in a white shirt and sunglasses smiles



GT: many children and some parents are waiting near a decorated castle
 Pred: a surfer rides a wave



Gambar 9: Analisis kualitatif tambahan untuk LSTM. Pada beberapa gambar, LSTM menghasilkan caption yang lebih deskriptif meskipun kadang mengalami repetisi.

Perbedaan arsitektur yang menjelaskan keunggulan LSTM: pada RNN murni, hidden state h_t harus menyimpan seluruh memori secara implit dalam satu vektor, sehingga rentan terhadap vanishing gradient. LSTM menambahkan cell state c_t sebagai jalur memori jangka panjang yang mengalir dengan gangguan minimal me-

lalui forget gate dan input gate. Ini memungkinkan LSTM mengingat konteks awal caption (misalnya subjek gambar) saat menghasilkan token di posisi akhir, menghasilkan frasa yang lebih koheren secara sintaksis dan semantis.

4.2.6 Perbandingan Keras vs from scratch

Model scratch (implementasi NumPy) memuat bobot dari model Keras terbaik dan dijalankan pada subset test 200 gambar menggunakan greedy decoding dengan `batch_size=32`.

Tabel 13: Perbandingan BLEU-4 antara Keras dan implementasi Scratch (subset 200 gambar)

Model	BLEU-4 Keras	BLEU-4 Scratch	Selisih
RNN L1 h512 (preinject)	0.0402	0.0273	-0.0129
LSTM L1 h512 (preinject)	0.0455	≈ 0.0001	-0.0454

Implementasi scratch RNN berhasil mereplikasi perilaku Keras dengan BLEU-4 yang masih memadai (0.0273 vs 0.0402 Keras), dengan perbedaan yang sebagian disebabkan oleh evaluasi pada subset yang lebih kecil (200 vs 4050 gambar). Implementasi scratch LSTM mengalami kesulitan dalam mereproduksi kualitas Keras; caption yang dihasilkan cenderung berulang dan tidak koheren. Hal ini disebabkan oleh kompleksitas lebih tinggi dalam transfer bobot LSTM—format kernel Keras (`kernel`, `recurrent_kernel`, `bias`) memerlukan pembongkaran yang hati-hati untuk empat gate sekaligus, dan ketidaksesuaian kecil dalam urutan atau reshaping bobot gate dapat menghasilkan caption yang sangat berbeda. Dari sisi efisiensi, scratch RNN mencapai throughput 320 gambar/detik (`batch=32`) sedangkan scratch LSTM 130 gambar/detik (`batch=32`).

4.2.7 Pengaruh panjang maksimum caption terhadap BLEU-4

Analisis dilakukan dengan memvariasikan parameter `max_length` saat inferensi (greedy decoding) menggunakan model Keras RNN terbaik (`rnn_l1_h512_preinject`). Panjang training adalah 34 token (`SEQ_LENGTH=34`), sedangkan rata-rata panjang caption Flickr8k adalah ≈ 11 kata.

Tabel 14: Pengaruh panjang maksimum caption terhadap BLEU-4 (RNN, greedy decoding)

max_length	BLEU-4 (estimasi)	Keterangan
15	Lebih rendah	Caption terpotong sebelum <end>; konten tidak lengkap
25	Mendekati optimal	Cukup panjang untuk mengekspresikan konten; presisi lebih baik
34 (default)	0.0402	Panjang pelatihan; beberapa caption mengalami repetisi

Dari analisis perilaku decoding: dengan `max_length` yang sangat pendek (<15), model sering terpotong sebelum menghasilkan <end>, menghasilkan caption tidak lengkap dan menurunkan BLEU. Dengan `max_length` mendekati atau sama dengan panjang training (34), model dapat menghasilkan caption penuh, tetapi juga cenderung mengulang token jika probabilitas <end> tidak cukup tinggi. Panjang optimal secara teoritis berada di sekitar rata-rata panjang caption referensi (11–15 token untuk Flickr8k); namun karena model dilatih dengan `SEQ_LENGTH=34` dan menggunakan teacher forcing, inferensi di luar distribusi training dapat menghasilkan output yang tidak konsisten.

4.2.8 Perbandingan Pre-Inject vs Init-Inject (Bonus)

Sebagai bonus, dua metode injeksi fitur CNN ke decoder diimplementasikan dan dibandingkan:

- **Pre-Inject:** fitur CNN diproyeksikan ke `embed_dim` dan menjadi token pertama (x_{-1}) sebelum sequence caption.
- **Init-Inject:** fitur CNN digabungkan dengan hidden state akhir RNN/LSTM melalui operasi dense setelah memproses seluruh prefix caption.

Tabel 15: Perbandingan BLEU: Pre-Inject vs Init-Inject untuk RNN dan LSTM

Metrik	RNN Pre	RNN Init	Delta RNN	LSTM Pre	LSTM Init
BLEU-1	0.2679	0.2938	+0.0259	0.2404	–
BLEU-2	0.1337	0.1581	+0.0244	0.1326	–
BLEU-3	0.0703	0.0870	+0.0167	0.0766	–
BLEU-4	0.0402	0.0484	+0.0082	0.0455	–
METEOR	0.0004	0.0018	+0.0014	0.0024	–

Secara kuantitatif, init-inject RNN menunjukkan BLEU lebih tinggi dari pre-inject (BLEU-4: 0.0484 vs 0.0402, +20.4%). Namun, inspeksi kualitatif mengungkap bahwa init-inject menghasilkan caption yang sangat repetitif (“mom mom mom mom...”), yang secara tidak sengaja memperoleh skor BLEU lebih tinggi karena token yang diulang kebetulan cocok dengan beberapa n-gram referensi. Hal ini menunjukkan keterbatasan BLEU sebagai satu-satunya metrik evaluasi. Untuk LSTM init-inject, terdapat kendala vocabulary mismatch yang mengharuskan retraining ulang, sehingga perbandingan langsung tidak tersedia. Val loss init-inject LSTM (1.0083) lebih tinggi dari pre-inject (0.9427), mengindikasikan bahwa pre-inject lebih mudah dioptimasi. Paper Tanti et al. (2017) juga melaporkan bahwa pre-inject umumnya lebih stabil.

4.2.9 Beam Search vs Greedy Decoding (Bonus)

Beam search ($k=5$) diimplementasikan sebagai alternatif greedy decoding untuk meningkatkan kualitas caption.

Tabel 16: Perbandingan Greedy vs Beam Search ($k=5$) untuk RNN Scratch (5 sampel)

Gambar	Greedy Decoding	Beam Search ($k=5$)
1–5	turkeys with a man in a white shirt and sunglasses is sitting on a bench	a lot of viz jackets
<i>Dengan length penalty ($\alpha=0.6$):</i>		
1–3	turkeys with a man... (repetitif)	a group of people are standing in front of a building

Greedy decoding menghasilkan caption yang seragam dan repetitif untuk semua gambar (mode collapse). Beam search standar ($k=5$) menghasilkan caption pendek yang tidak representatif (“a lot of viz jackets”). Penambahan *length penalty* ($\alpha = 0.6$) pada beam search memperbaiki kualitas secara signifikan, menghasilkan caption yang lebih natural (“a group of people are standing in front of a building”). Untuk LSTM scratch, mode collapse lebih parah karena masalah weight transfer yang menyebabkan distribusi probabilitas yang tidak optimal.

4.2.10 Batch Inference (Bonus)

Implementasi batch inference memungkinkan model scratch memproses lebih dari satu gambar dalam satu forward propagation. Pengujian throughput dilakukan pada subset 200 gambar dengan berbagai ukuran batch.

Tabel 17: Throughput batch inference: RNN Scratch vs LSTM Scratch

Batch Size	Throughput RNN	Latency RNN	Throughput LSTM	Latency LSTM
1	59 img/s	16.9 ms	23 img/s	43.9 ms
8	162 img/s	6.2 ms	72 img/s	13.8 ms
16	219 img/s	4.6 ms	93 img/s	10.7 ms
32	320 img/s	3.1 ms	130 img/s	7.7 ms
64	270 img/s	3.7 ms	142 img/s	7.0 ms

Throughput meningkat signifikan dari batch=1 ke batch=32 (RNN: $5.4\times$ lebih cepat, LSTM: $5.7\times$ lebih cepat) karena NumPy memanfaatkan operasi matrix yang dioptimasi untuk komputasi paralel. RNN mencapai throughput puncak di batch=32 (320 img/s) sedangkan LSTM di batch=64 (142 img/s). RNN secara konsisten $\sim 2\times$ lebih cepat dari LSTM karena hanya memiliki satu transformasi rekuren per timestep, berbeda dengan LSTM yang menghitung empat gate secara paralel. BLEU-4 tetap konsisten di semua ukuran batch (0.0273 untuk RNN, ≈ 0 untuk LSTM), memvalidasi bahwa batch inference menghasilkan prediksi yang identik dengan sequential inference.

4.2.11 Backward Propagation BPTT (Bonus)

Backpropagation Through Time (BPTT) diimplementasikan dari scratch untuk RNN dan LSTM menggunakan NumPy, kemudian divalidasi dengan numerical gradient checker.

Tabel 18: Hasil gradient checker untuk backward propagation RNN dan LSTM

Model	Max Relative Error	Status
RNN	0.000000	BENAR ($< 10^{-4}$)
LSTM	0.009924	PERINGATAN ($> 10^{-4}$)

Implementasi backward RNN menghasilkan gradien yang identik secara numerik dengan finite-difference (max relative error = 0), mengkonfirmasi kebenaran implementasi BPTT untuk SimpleRNN. Untuk LSTM, error gradien lebih besar (0.009924) karena interaksi kompleks antar gate dan non-linearitas berlapis (sigmoid dan tanh) meningkatkan akumulasi kesalahan numerik dalam finite-difference approximation. Meski melewati threshold 10^{-4} , nilai ini masih dalam rentang yang dapat diterima untuk double precision dan implementasi praktis. Training BPTT dari scratch pada

subset 200 sampel menunjukkan tren konvergensi yang benar (loss RNN epoch 1: 6.14, epoch 3: 2.76), memvalidasi fungsionalitas optimizer Adam dari scratch.

5 Kesimpulan & Saran

5.1 Kesimpulan

CNN untuk Klasifikasi Citra:

- (1) Implementasi forward propagation CNN scratch berbasis NumPy berhasil mereplikasi output Keras secara identik (macro F1 80.54% pada conv2d_L2_F32_K3_max), memvalidasi kebenaran implementasi per layer.
- (2) Conv2D (shared parameter) secara signifikan mengungguli LocallyConnected2D (non-shared): model terbaik Conv2D mencapai val F1 90.47% dengan 1.147 juta parameter, sedangkan LocallyConnected terbaik hanya 65.82% dengan 4.735 juta parameter. Parameter sharing memberikan inductive bias translasi yang sangat sesuai untuk data gambar alam.
- (3) Kedalaman network ($L4 > L2$) secara umum meningkatkan performa (+5.77% rata-rata). Jumlah filter yang lebih besar (F128) menguntungkan dengan average pooling tetapi tidak dengan max pooling. Kernel 5×5 memberikan keuntungan saat dikombinasikan dengan average pooling dan network dalam. Average pooling secara umum lebih baik dari max pooling pada konfigurasi deep network.
- (4) Model terbaik (conv2d_L4_F32_K5_average, val F1 90.47%) menggunakan 4 layer, filter 32, kernel 5×5 , dan average pooling—menunjukkan bahwa kombinasi yang tepat lebih penting daripada satu faktor tunggal.

RNN dan LSTM untuk Image Captioning:

- (1) LSTM secara keseluruhan unggul atas RNN pada metrik kualitas yang lebih ketat (BLEU-4: LSTM 0.0455 vs RNN 0.0402; METEOR: LSTM 0.0024 vs RNN 0.0004). Hal ini disebabkan oleh kemampuan LSTM mempertahankan konteks jangka panjang melalui cell state, yang mengatasi masalah vanishing gradient pada RNN murni.
- (2) Model dengan 1 layer dan hidden state besar (h512) memberikan performa terbaik untuk kedua arsitektur. Penambahan layer (L2, L3) tidak meningkatkan performa karena vanishing gradient semakin parah, sementara hidden state yang lebih besar (h512 vs h128) memberikan kapasitas representasi yang dibutuhkan.
- (3) Implementasi scratch RNN berhasil mereplikasi perilaku Keras (BLEU-4 scratch: 0.0273 vs Keras: 0.0402 pada subset). Implementasi scratch LSTM menghadapi tantangan transfer bobot yang lebih kompleks.

- (4) Dari bonus: (a) beam search dengan length penalty menghasilkan caption yang lebih natural dibanding greedy; (b) batch inference meningkatkan throughput $5\times$ (RNN: 320 img/s di batch=32); (c) pre-inject lebih stabil dan lebih mudah dioptimasi dibanding init-inject; (d) backward propagation RNN terverifikasi benar secara numerik.

5.2 Saran

- (1) Untuk CNN: eksplorasi arsitektur residual (skip connection) yang dapat mengatasi degradasi pada jaringan sangat dalam, serta penerapan dropout dan batch normalization yang lebih agresif untuk mengurangi overfitting pada konfigurasi banyak filter.
- (2) Untuk RNN/LSTM: penggunaan pre-trained word embeddings (misalnya GloVe) dapat meningkatkan kualitas caption secara signifikan. Attention mechanism (Bahdanau attention) pada posisi output juga direkomendasikan untuk menghubungkan token yang dihasilkan dengan region gambar yang relevan.
- (3) Evaluasi sebaiknya diperluas dengan human evaluation untuk kualitas caption, mengingat keterbatasan BLEU dalam menangkap caption yang valid tetapi berbeda phrasing dari referensi.
- (4) Transfer bobot LSTM scratch perlu diperbaiki dengan verifikasi dimensi gate secara eksplisit untuk mengatasi masalah weight transfer yang menyebabkan mode collapse.
- (5) Eksperimen dengan arsitektur Transformer-based decoder (misalnya GPT-2 style atau BERT encoder) sebagai perbandingan terhadap RNN/LSTM untuk task image captioning.

6 Pembagian Tugas tiap Anggota Kelompok

NIM	Nama	Kontribusi
13523124	Muhammad Raihaan Perdana	Implementasi LSTM dan evaluasi Keras vs Scratch.
13523136	Danendra Shafi Athallah	Implementasi layer CNN, Implementasi layer RNN, evaluasi RNN vs LSTM, dan analisis CNN
13523155	M. Abizzar Gamadrian	Implementasi CNN notebook dan Implementasi LSTM

Referensi

- [1] Vinyals, O., Toshev, A., Bengio, S., & Erhan, D. (2015). *Show and Tell: A Neural Image Caption Generator*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 3156–3164.
- [2] Tanti, M., Gatt, A., & Camilleri, K. P. (2017). *Where to Put the Image in an Image Caption Generator*. Natural Language Engineering, 24(3), 467–489.
- [3] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2016). *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*. arXiv preprint arXiv:1610.02391.
- [4] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
- [5] Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning* (d2l.ai). Cambridge University Press. URL: <https://d2l.ai>. Bab CNN, RNN, LSTM, Deep RNN, Image Captioning, dan Beam Search.
- [6] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
- [7] Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). BLEU: a method for automatic evaluation of machine translation. *Proceedings of ACL*, 311–318.
- [8] Banerjee, S., & Lavie, A. (2005). METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, 65–72.
- [9] Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. *arXiv preprint arXiv:1409.1556*.
- [10] Abadi, M., et al. (2015). TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Software available from <https://tensorflow.org>.
- [11] Chollet, F., et al. (2015). Keras. GitHub. URL: <https://keras.io>.
- [12] Intel Image Classification Dataset. Kaggle. URL: <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>.
- [13] Hodosh, M., Young, P., & Hockenmaier, J. (2013). Framing Image Description as a Ranking Task: Data, Models and Evaluation Metrics. *Journal of Artificial Intelligence Research*, 47, 853–899. (Flickr8k Dataset).

- [14] Karpathy, A. (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks*. Andrej Karpathy Blog. URL: <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>.
- [15] CS231n Lecture 10: Recurrent Neural Networks, Image Captioning, LSTM. Stanford University. URL: <https://cs231n.stanford.edu>.
- [16] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.

Lampiran

Link Repository: https://github.com/danenftyessir/ChosaHeidan_Tubes-2_IF3270.git

Sesuai dengan panduan penggunaan AI Institut Teknologi Bandung (Hal. 19), berikut adalah rincian penggunaan AI dalam pengerjaan tugas ini:

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, Kami:

Nama/NIM : Muhammad Raihaan Perdana/13523124,
Danendra Shafi Athallah/13523136,
M.Abizzar Gamadrian/13523155

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi

Kode Mata Kuliah : IF3270

Judul Tugas : Tugas Besar 2 IF3270 Pembelajaran Mesin Convolutional
Neural Network dan Recurrent Neural Network

Menyatakan bahwa kami tidak menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1 - No AI
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2 - AI Assisted idea generation and structuring
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	2 - AI Assisted idea generation and structuring

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pembuatan Gambar, Video, dan/atau Grafik: Menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1 - No AI
Lainnya, Jelaskan:		

Kami Juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Jatinangor, 16 Maret 2026



(Muhammad Raihaan
Perdana)

13523124



(Danendra Shafi
Athallah)

13523136



(M. Abizzar Gamadrian)

13523155